

Practical guidelines for harness engineering with Claude Code.

Seven categories of control around the model, and how to build them in a real code base.

Today

- | | | |
|----|---|---------|
| 01 | What is harness engineering?
<i>... and why do we care</i> | 03 - 05 |
| 02 | Seven categories of harness components
 | 06 - 13 |
| 03 | Latest trends | 14 |
| 04 | Demo | live |

Agent = Model + Harness

"Harness engineering" coined by **Mitchell Hashimoto** · Feb 2026
Equation shaped by **Vivek Trivedy** at LangChain · shortly after

Why is this important?

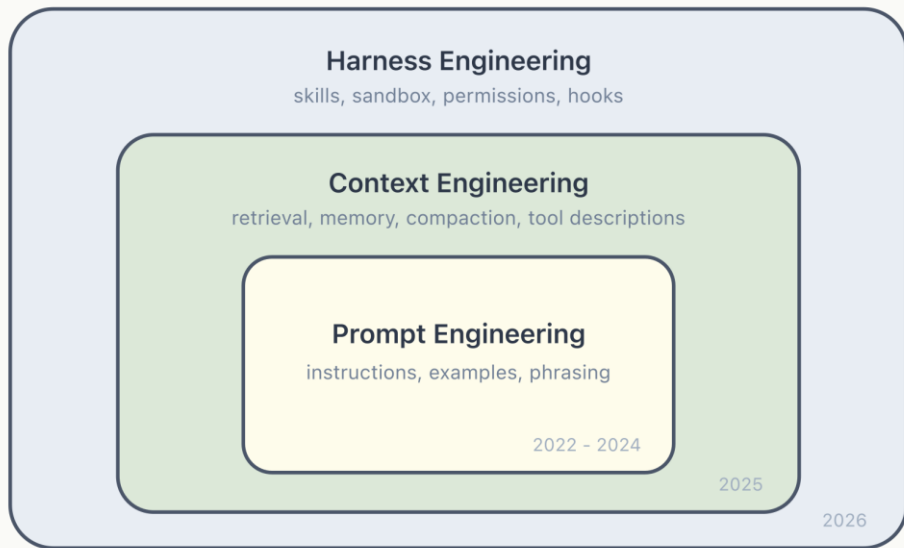
We are concerned about agentic code quality — and do not simply rely on prompting (vibe coding).

52.8% → 66.5% TerminalBench, harness only

42% → 78% CORE-Bench, Opus in Claude Code

Three eras of engineering leverage

Each discipline subsumes the ones before it. You don't throw earlier work away — you embed it.



WHY THE NESTING MATTERS

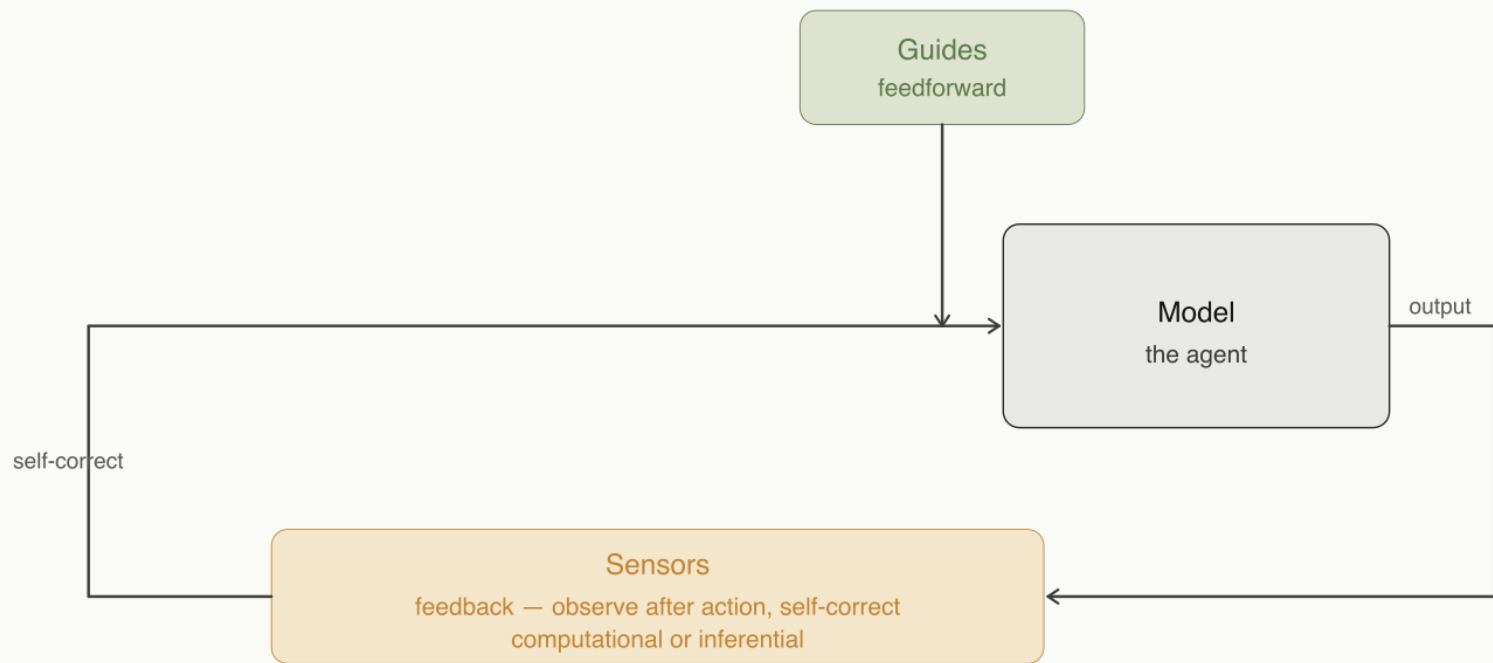
A good harness *contains* good context engineering.

A good context window *contains* good prompting.

Once the model is capable enough, the bottleneck migrates outward — to the entire execution environment around the loop.

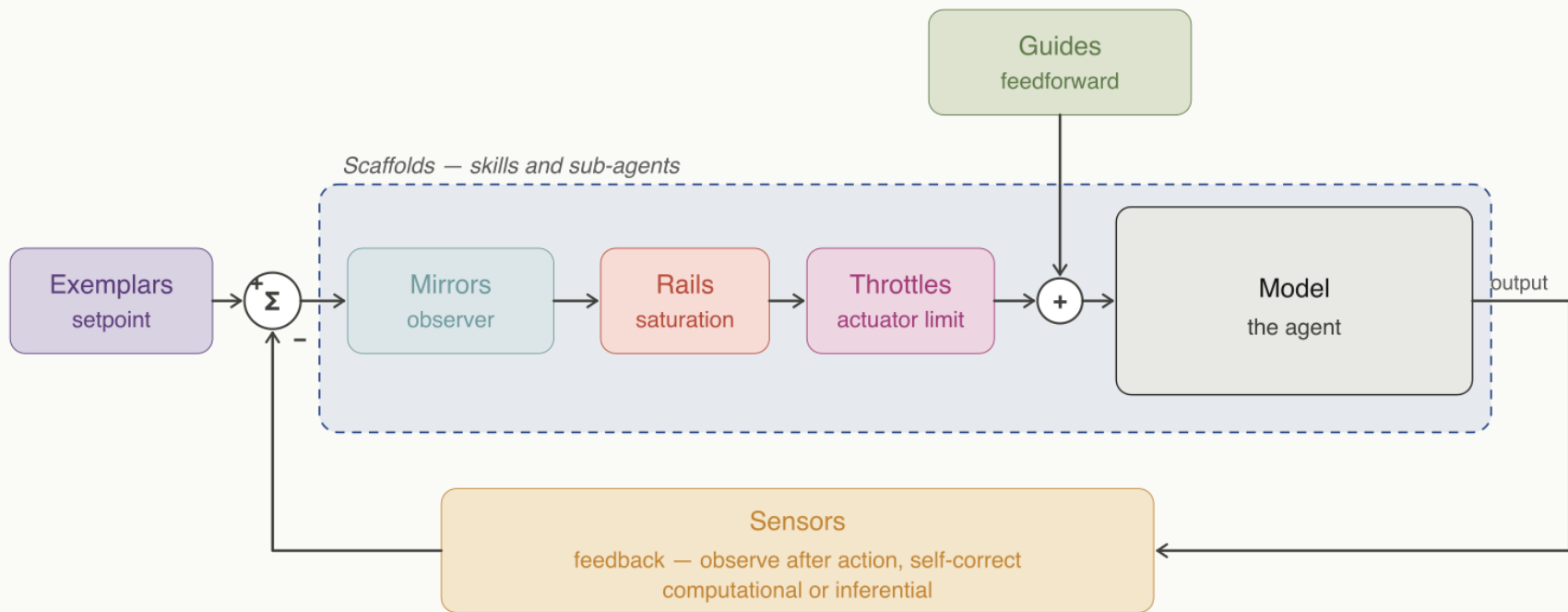
Böckeler: Guides and sensors

Feedforward (guides) and Feedback (sensors) are terms from Control Theory



Seven categories of harness component

A framework for applied harness engineering inspired by control theory



Guides

./CLAUDE.md – KEPT SHORT, SPECIFIC, FALSIFIABLE

Project conventions

- Stack: FastAPI + PostgreSQL 16.
- Tests: pytest, not unittest.
- HTTP clients: httpx, not requests.
- Branch off staging. Never commit to main.

Commands

```
just test      # unit tests
just check     # ruff + ty + pytest
just migrate   # alembic upgrade head
```

Iron laws

- Never git push --force to main.
- Never hand-edit alembic/versions/.

Bias what the agent produces
before it acts.

A top-level CLAUDE.md, optional per-directory files for subsystems with their own conventions, and the system prompt itself.

RULE OF THUMB

Keep it short — less than 200 lines.
Hand-author your conventions.
Prune ruthlessly.

Sensors

POSTTOOLUSE HOOK — ERROR SIGNAL FLOWS BACK INTO AGENT

```
{
  "hooks": {
    "PostToolUse": [{
      "matcher": "Edit|Write",
      "hooks": [{
        "type": "command",
        "command": "ruff format $F && ty $F"
      }]
    }]
  }
}
```

inferential sensors — examples

Screenshots — agent judges the layout.

Reviewer sub-agent — judges the diff.

Runtime logs — does the warning matter?

Observe what the agent did.

Return an error signal.

Without sensors the agent runs open-loop.

Computational — deterministic: linters, test runners.

Inferential — agent-interpreted (Böckeler & Ford).

RULE OF THUMB

Scope each sensor to the blast radius of the change.

Silent on success.

Noisy on failure.

Output failure message in JSON format.

(preferable in fixed envelope shape)

Rails

DENY ENTRIES – THE ONLY LAYER WHERE "CANNOT" IS LITERAL

```
{  
  "permissions": {  
    "deny": [  
      "Bash(rm -rf *)",  
      "Bash(git push --force*)",  
      "Bash(psql * DROP DATABASE*)",  
      "Bash(DATABASE_URL=*prod* *)"  
    ],  
    "allow": [  
      "Bash(just test*)",  
      "Bash(just check*)"  
    ]  
  }  
}
```

Invariants in CLAUDE.md are wishes.
Invariants in permissions are laws.

RULE OF THUMB

If an invariant matters, it goes here.

Not in the prose.

Not in a comment.

In the deny list.

Scaffolds

skill – packaged procedure

```
# .claude/skills/release/
---
name: release
description: Cut and tag
  release after PR merge
---
1. just check
2. bump pyproject.toml
3. git tag && push --tags
```

sub-agent – critic

```
# .claude/agents/reviewer.md
---
name: reviewer
description: Diff-only
  critic, isolated context
tools: Read, Grep,
  Bash(git diff*)
---
Review the diff. Never open
files outside it.
```

workflow – fan-out

```
# .claude/workflows/review.js
const DIRS =
  ["api", "web", "jobs"]

const bugs = await parallel(
  DIRS.map(d => () => agent(
    `Find bugs in ${d}`)))

const real = await parallel(
  bugs.map(b => () => agent(
    `Try to refute: ${b}`)))

return real.filter(Boolean)
```

Skills package procedures. Sub-agents isolate context. Workflows make fan-out deterministic.

Exemplars

Pointers, not prose

```
# CLAUDE.md – exemplars section
```

When adding a new HTTP endpoint:

- `routes/invoices.py` – req/resp shape
- `routes/test_invoices.py` – fixture style

When adding a new background job:

- `jobs/reminder_emails.py` – retry, idempotent

Not "write Pythonic Python." A pointer to a specific file as the canonical shape.

A worked example is worth a page of prose.

The agent pattern-matches to a trajectory instead of reasoning toward one. And exemplars are cheap — you're already writing the code.

RULE OF THUMB

Keep the pointers fresh. A reference file the agent dutifully copies from long after the real convention has moved on is worse than no exemplar.

Mirrors

Two mechanisms, same purpose: spend tokens on reflection before spending tokens on action.

Plan mode — when scope isn't obvious. Forces a written plan before any Edit/Write/Bash.

Extended thinking — when the first answer is usually wrong.

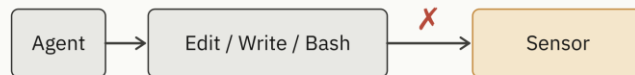
RULE OF THUMB

Reach when the failure mode is confident wrongness.

Not when information is missing.

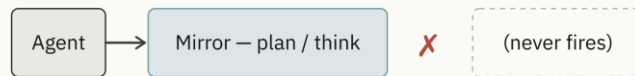
Skip mirrors for mechanical tasks.

WITHOUT MIRROR



caught AFTER the tool fires

WITH MIRROR



caught BEFORE the tool fires

Sensors catch wrongness *after* a tool call.

Mirrors catch it *before*.

Throttles

Decide the bounds in advance.

An autonomous run, bounded

```
$ claude -p \  
  --max-turns 25 \  
  --max-budget-usd 5.00 \  
  --permission-mode acceptEdits \  
  "Implement the refund flow."
```

Three bounds, three layers

ITERATION `--max-turns` CLI

SPEND `--max-budget-usd` CLI (-p) · Stop hook

AUTHORITY `--permission-mode` CLI · settings.json

`--max-budget-usd` is a print-mode-only flag — interactive sessions need a Stop hook that parses the transcript and emits `{"continue": false}` when the budget is exceeded.

Where the field is pushing

Two buzzwords that dissolve into the seven categories — and three gaps that don't.

the buzzword check

“Loop engineering”?

“Graph engineering”?

Useful lenses, not new disciplines — both map onto the existing categories:

GRAPHS → **Scaffolds** (process topology)

LOOPS → **Sensors** · **Scaffolds** · **Throttles**
feedback · topology · termination

three gaps the newer taxonomies expose

Memory

Persistent state across sessions — 8 dedicated failure modes in Scale AI’s “Model or Harness?” (Jul 2026).

Verification

A full layer in the 17-author ETCLOVG survey — the field’s answer to “how to evaluate harness changes?”

Attenuation

Filtering agent output toward the human (Thoughtworks) — all seven categories point at the agent.

Stop wishing for a better LLM

Work on improving your harness setup

Walk the seven categories. Where are you lopsided?

Evaluation is arriving — Scale AI’s “Model or Harness?”: a failure is model-side if a stronger model would have avoided it.

Thanks for listening

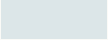
01   Guides

02   Sensors

03   Rails

04   Scaffolds

05   Exemplars

06   Mirrors

07   Throttles

All details — including extensive references — at:

harness-engineering.carstenj.workers.dev

Questions? Let's connect on LinkedIn:

linkedin.com/in/carstenjoergensen